

Operating Systems (CS 2006)

Final Exam

Date: May 21, 2026

Course Instructor(s)

Dr.Ghufran, Minhal Raza, Saeeda Kanwal, Rabia Ahmed,
Anaum Hamid,UbaidUllah, Zulfiqar Ali,Syed Faisal Ali

Total Time (Hrs.): 3

Total Marks: 100 Points

Total Questions: 6

Roll No

Section

Student Signature

Attempt all questions in Order

Instruction:

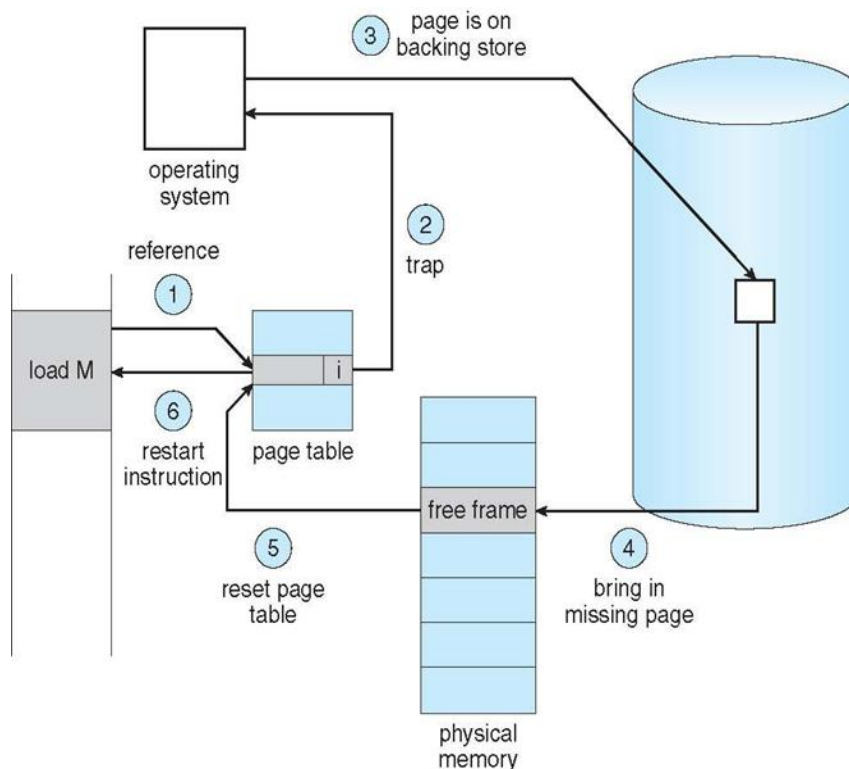
Avoid unnecessary details in your answers. Try to write answers as bullet points. Make suitable assumptions, if needed. Return the question paper. Students are not allowed to write anything on the question paper except roll number & section. Read each question completely before answering it. There are **6 questions and 4 pages**.

CLO # 5 - Understand the deadlock and memory management.

Q1 [a(8) + b(8)+ c(5)+ d(8) = 29 Points | 50 Mins]

a)

- i. **Explain Page fault processing using a labelled diagram. [4]**



National University of Computer and Emerging Sciences

Karachi Campus

- ii. A specialized server uses a demand-paging memory system. The hardware memory access time is 150 nanoseconds, and servicing a page fault takes an average of 6 milliseconds. If exactly one out of every 500 memory accesses causes a page fault, calculate the Effective Access Time (EAT) of this system. Show your formula, unit conversions, and calculations. [4]

Effective Access Time (EAT) Calculation – Demand Paging System

- Hardware memory access time = 150 nanoseconds (ns)
- Average time to service a page fault = 6 milliseconds (ms)
- Page fault rate = 1 out of every 500 memory accesses

Formula:

$$EAT = (1 - p) \times T_{mem} + p \times T_{pf}$$

Where:

p = page fault rate

T_{mem} = memory access time (no page fault)

T_{pf} = page fault service time

Step 1: Convert units to the same unit (nanoseconds)

$$1 \text{ ms} = 1,000,000 \text{ ns}$$

$$\therefore 6 \text{ ms} = 6 \times 1,000,000 \text{ ns} \\ = 6,000,000 \text{ ns}$$

Page fault probability:

$$p = \frac{1}{500} = 0.002$$

Probability of normal access:

$$1 - p = 1 - 0.002 = 0.998$$

Step 3: Perform the calculations

Component	Calculation	Result (ns)
Normal memory access (no page fault)	$0.998 \times 150 \text{ ns}$	149.7 ns
Page fault case (service time)	$0.002 \times 6,000,000 \text{ ns}$	12,000 ns
Total EAT	$149.7 \text{ ns} + 12,000 \text{ ns}$	12,149.7 ns

Step 2: Substitute values into the formula

$$EAT = (1 - p) \times T_{mem} + p \times T_{pf} \\ = (0.998 \times 150 \text{ ns}) + (0.002 \times 6,000,000 \text{ ns})$$

Step 4: Final Answer

$$EAT = 12,149.7 \text{ ns}$$

Convert to microseconds (μs):

$$12,149.7 \text{ ns} \div 1,000 = 12.1497 \mu\text{s}$$

Effective Access Time

$$\approx 12,150 \text{ ns}$$

$$(\approx 12.15 \mu\text{s})$$

b) A system is testing its memory management under a heavy looping workload.

- i. You are tasked with comparing the performance of LRU and second chance algorithm for 3 Page frames configurations [3+3 = 6]

The Reference String (20 requests):

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3

Reference	Frame 1	Frame 2	Frame 3	Fault/Hit
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
4	4	2	3	Fault
1	4	1	3	Fault
2	4	1	2	Fault
5	5	1	2	Fault
1	5	1	2	Hit
2	5	1	2	Hit
3	3	1	2	Fault
4	3	4	2	Fault
5	3	4	5	Fault
1	1	4	5	Fault
2	1	2	5	Fault
3	1	2	3	Fault

Reference	Frame 1	Frame 2	Frame 3	Fault/Hit
4	4	2	3	Fault
5	4	5	3	Fault
1	4	5	1	Fault
2	2	5	1	Fault
3	2	3	1	Fault

National University of Computer and Emerging Sciences

Karachi Campus

SECOND CHANCE (CLOCK) PAGE REPLACEMENT 3 FRAMES						
REFERENCE STRING (20 Requests): 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3						
Frames = 3 Pointer starts at Frame 0						

RULES (SECOND CHANCE / CLOCK)

- On a new page insert → reference bit = 0
- On a hit (page found) → set reference bit = 1
- For replacement, use CLOCK (circular) order:
 - If reference bit = 1 → set it to 0 and move on
 - If reference bit = 0 → replace that page

Step	Reference	Hit / Fault	Frames (Page / Reference Bit)			Pointer Position	Action / Explanation
			F0	F1	F2		
1	1	PF	1(0)	-	-	F1	Insert 1 in F0
2	2	PF	1(0)	2(0)	-	F2	Insert 2 in F1
3	3	PF	1(0)	2(0)	3(0)	F0	Insert 3 in F2
4	4	PF	4(0)	2(0)	3(0)	F1	F0 had 0 → replace 1 with 4
5	1	PF	4(0)	1(0)	3(0)	F2	F1 had 0 → replace 2 with 1
6	2	PF	4(0)	1(0)	2(0)	F0	F2 had 0 → replace 3 with 2
7	5	PF	5(0)	1(0)	2(0)	F1	F0 had 0 → replace 4 with 5
8	1	H	5(0)	1(1)	2(0)	F1	Hit in F1 → set bit = 1
9	2	H	5(0)	1(1)	2(1)	F1	Hit in F2 → set bit = 1
10	3	PF	3(0)	1(0)	2(0)	F1	F1:1→0, F2:1→0, F0:0 → replace 5 with 3
11	4	PF	3(0)	4(0)	2(0)	F2	F1 had 0 → replace 1 with 4
12	5	PF	3(0)	4(0)	5(0)	F0	F2 had 0 → replace 2 with 5
13	1	PF	1(0)	4(0)	5(0)	F1	F0 had 0 → replace 3 with 1
14	2	PF	1(0)	2(0)	5(0)	F2	F1 had 0 → replace 4 with 2
15	3	PF	1(0)	2(0)	3(0)	F0	F2 had 0 → replace 5 with 3
16	4	PF	4(0)	2(0)	3(0)	F1	F0 had 0 → replace 1 with 4
17	5	PF	4(0)	5(0)	3(0)	F2	F1 had 0 → replace 2 with 5
18	1	PF	4(0)	5(0)	1(0)	F0	F2 had 0 → replace 3 with 1
19	2	PF	2(0)	5(0)	1(0)	F1	F0 had 0 → replace 4 with 2
20	3	PF	2(0)	3(0)	1(0)	F2	F1 had 0 → replace 5 with 3

LEGEND

PF → Page Fault
H → Hit
R → Reference (Second Chance) Bit (0 or 1)

CLOCK (Circular Frames)

EXPLANATION OF STEPS

When a page fault occurs, we start checking from the frame pointed by the pointer.

- If R Bit = 1 → give second chance (set R Bit = 0) and move to next frame.
- If R Bit = 0 → replace that page with new page (R Bit of new page set to 0).

Pointer always moves to next frame (circularly).

SUMMARY

Total References = 20
Total Hits = 2
Total Page Faults = 18
Hit Ratio = 2 / 20 = 10%
Fault Ratio = 18 / 20 = 90%

FINAL FRAME STATE	F0	F1	F2
	2 (0)	3 (0)	1 (0)

→ Pointer is at Frame 2

- ii. Does the performance improve with more memory? Does it suffer from Belady's Anomaly? [1]
 LRU (Least Recently Used) and Second Chance algorithms generally perform better when more memory frames are available because page replacements occur less frequently, which reduces page faults. However, LRU does not suffer from Belady's Anomaly since it is a stack algorithm, meaning adding more memory

National University of Computer and Emerging Sciences

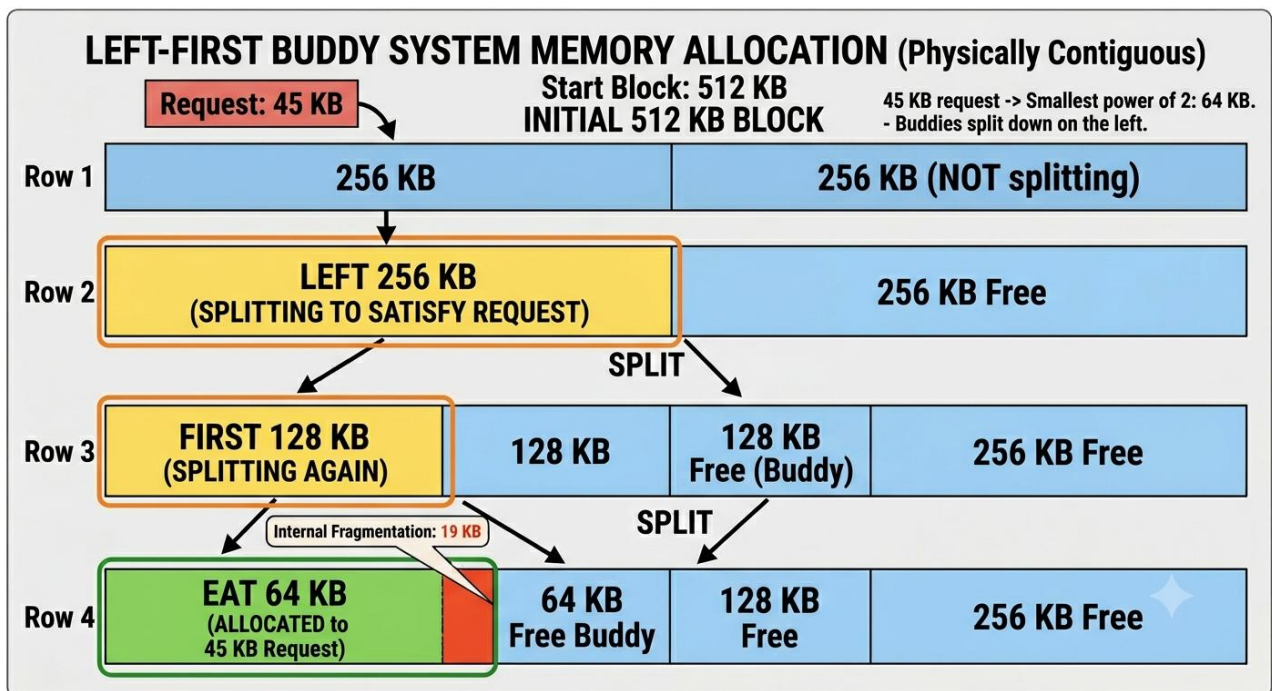
Karachi Campus

never increases page faults. In contrast, Second Chance can suffer from Belady's Anomaly because it is only an approximation of LRU

iii. Comment on the performance of both LRU and Second Chance Algorithms. [1]

In terms of performance, LRU usually provides better page replacement decisions and lower page fault rates but requires more implementation overhead. Second Chance is simpler, less costly, and widely used in practice because it offers a good balance between efficiency and implementation complexity.

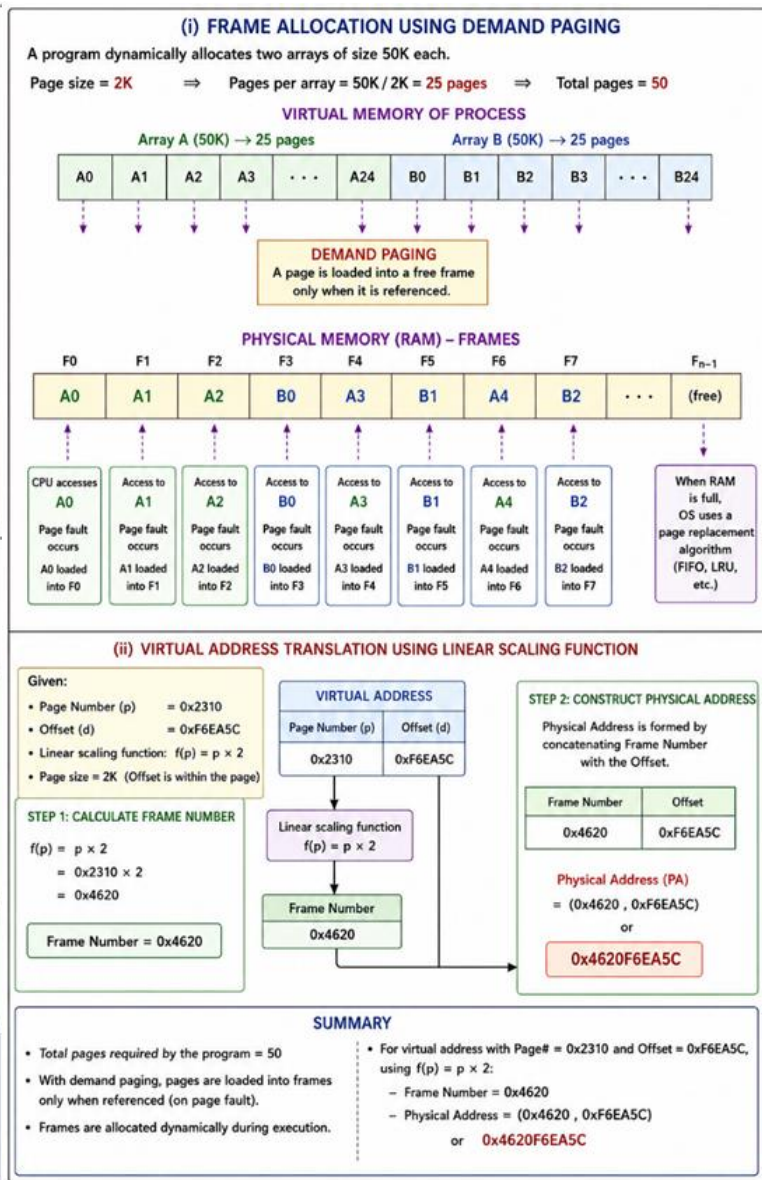
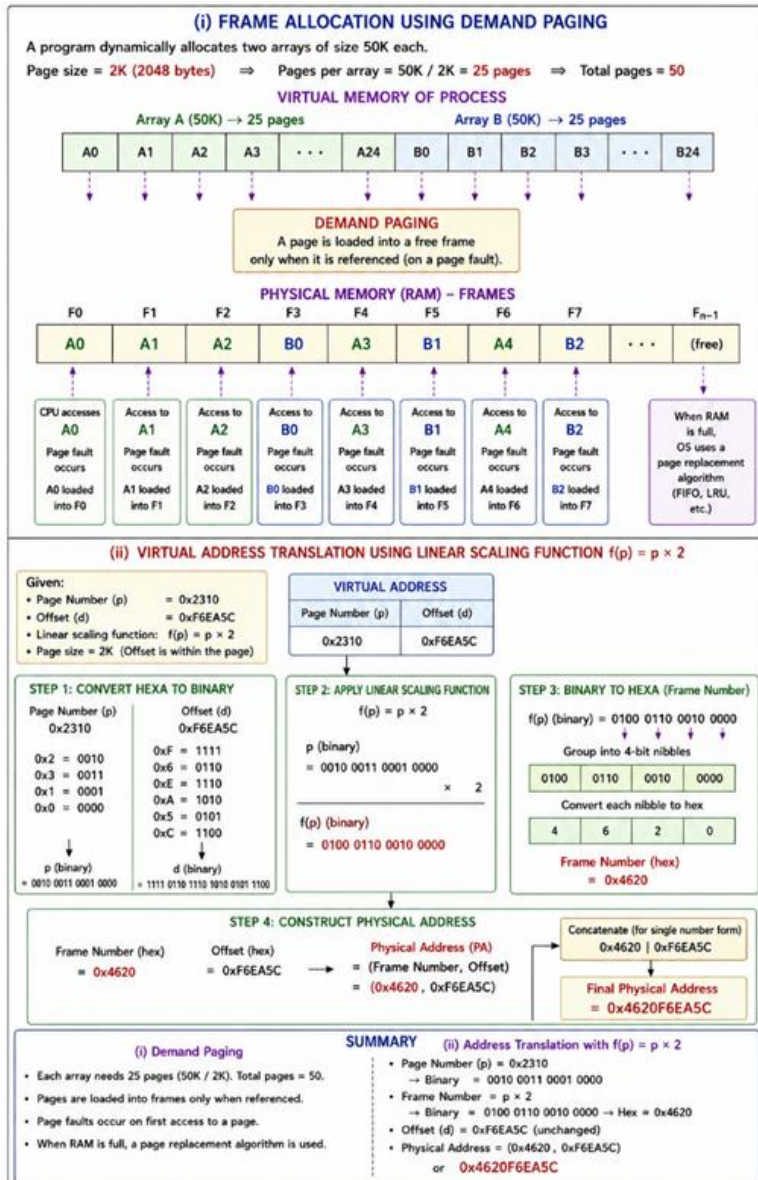
c) The kernel uses a Buddy System for memory allocation, starting with a physically contiguous segment of 512 KB. A request arrives for 45 KB of memory. Show how the blocks are split to satisfy this request. [5]



National University of Computer and Emerging Sciences

Karachi Campus

- d) Suppose a program that adds two dynamically allocated arrays of size 50K each in a virtual memory system using demand paging with page size 2K.
- Explain using a labelled diagram **how frames are allocated to this program during execution**. [4]
 - Demonstrate the step-by-step translation of a virtual address into a physical address, using page number 0x2310 and offset 0xF6EA5C by applying the linear scaling function, **frame number = page number * 2**, to mathematically derive the frame number without a page table lookup. [4]



CLO # 5 - Understand the deadlock and memory management.

Q2.[a(10)+b(10) = 20 Points | 35 Mins]

a) A file processing system has **three threads (T1, T2, T3)** and **three files (file-A, file-B, file-C)**. Each thread locks files in the following order:

- Thread T1 locks file-A, then tries to lock file-B.
- Thread T2 locks file-B, then tries to lock file-C.
- Thread T3 locks file-C, then tries to lock file-A.

Tasks:

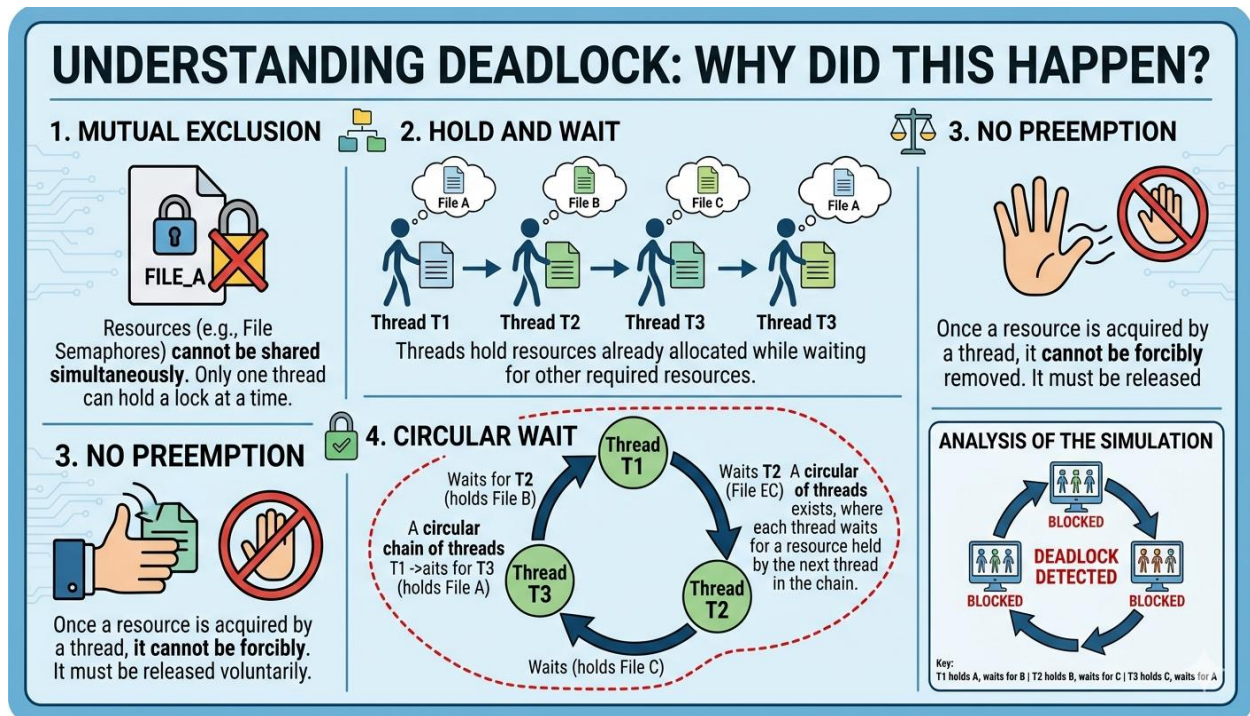
National University of Computer and Emerging Sciences

Karachi Campus

- i. Write a C program using semaphores that demonstrates this deadlock.[4]

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  // Define semaphores representing the files
8  sem_t file_A;
9  sem_t file_B;
10 sem_t file_C;
11
12 // Thread 1: Locks A, then tries to lock B
13 void* thread1_func(void* arg) {
14     printf("[Thread 1] Trying to lock File A...\n");
15     sem_wait(&file_A);
16     printf("[Thread 1] Successfully locked File A. Now working...\n");
17
18     // Sleep to guarantee other threads grab their first files, forcing the deadlock
19     sleep(1);
20
21     printf("[Thread 1] Trying to lock File B...\n");
22     sem_wait(&file_B); // Will block here indefinitely
23
24     // The following will never execute during a deadlock
25     printf("[Thread 1] Successfully locked File B. Processing...\n");
26     sem_post(&file_B);
27     sem_post(&file_A);
28     return NULL;
29 }
30
31 // Thread 2: Locks B, then tries to lock C
32 void* thread2_func(void* arg) {
33     printf("[Thread 2] Trying to lock File B...\n");
34     sem_wait(&file_B);
35     printf("[Thread 2] Successfully locked File B. Now working...\n");
36
37     sleep(1);
38
39     printf("[Thread 2] Trying to lock File C...\n");
40     sem_wait(&file_C); // Will block here indefinitely
41
42     // The following will never execute during a deadlock
43     printf("[Thread 2] Successfully locked File C. Processing...\n");
44     sem_post(&file_C);
45     sem_post(&file_B);
46     return NULL;
47 }
48
49 // Thread 3: Locks C, then tries to lock A
50 void* thread3_func(void* arg) {
51     printf("[Thread 3] Trying to lock File C...\n");
52     sem_wait(&file_C);
53     printf("[Thread 3] Successfully locked File C. Now working...\n");
54
55     sleep(1);
56
57     printf("[Thread 3] Trying to lock File A...\n");
58     sem_wait(&file_A); // Will block here indefinitely
59
60     // The following will never execute during a deadlock
61     printf("[Thread 3] Successfully locked File A. Processing...\n");
62     sem_post(&file_A);
63     sem_post(&file_C);
64     return NULL;
65 }
66
67 int main() {
68     pthread_t t1, t2, t3;
69
70     // Initialize semaphores with a value of 1 (acting as mutexes/binary semaphores)
71     sem_init(&file_A, 0, 1);
72     sem_init(&file_B, 0, 1);
73     sem_init(&file_C, 0, 1);
74
75     printf("=== Starting Deadlock Simulation ===\n");
76
77     // Create the threads
78     pthread_create(&t1, NULL, thread1_func, NULL);
79     pthread_create(&t2, NULL, thread2_func, NULL);
80     pthread_create(&t3, NULL, thread3_func, NULL);
81
82     // Wait for threads to finish (they never will)
83     pthread_join(t1, NULL);
84     pthread_join(t2, NULL);
85     pthread_join(t3, NULL);
86
87     // Clean up semaphores (unreachable code in deadlock state)
88     sem_destroy(&file_A);
89     sem_destroy(&file_B);
90     sem_destroy(&file_C);
91
92     return 0;
93 }
```


- ii. Identify and explain which of the four necessary conditions for deadlock are present in this scenario.[2]



- iii. Modify the code to prevent the deadlock and explain the strategy that you used for deadlock prevention.[4]

```

/* Deadlock-Free Code using Resource Ordering */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t fileA, fileB, fileC;

void* thread_func(void* arg) {
    int id = *(int*)arg;

    sem_wait(&fileA);
    printf("T%d locked file-A\n", id);
    sleep(1);

    sem_wait(&fileB);
    printf("T%d locked file-B\n", id);
    sleep(1);
    sem_wait(&fileC);
    printf("T%d locked file-C\n", id);

    sem_post(&fileC);
    sem_post(&fileB);
    sem_post(&fileA);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    int id1 = 1, id2 = 2, id3 = 3;

    sem_init(&fileA, 0, 1);
    sem_init(&fileB, 0, 1);
    sem_init(&fileC, 0, 1);

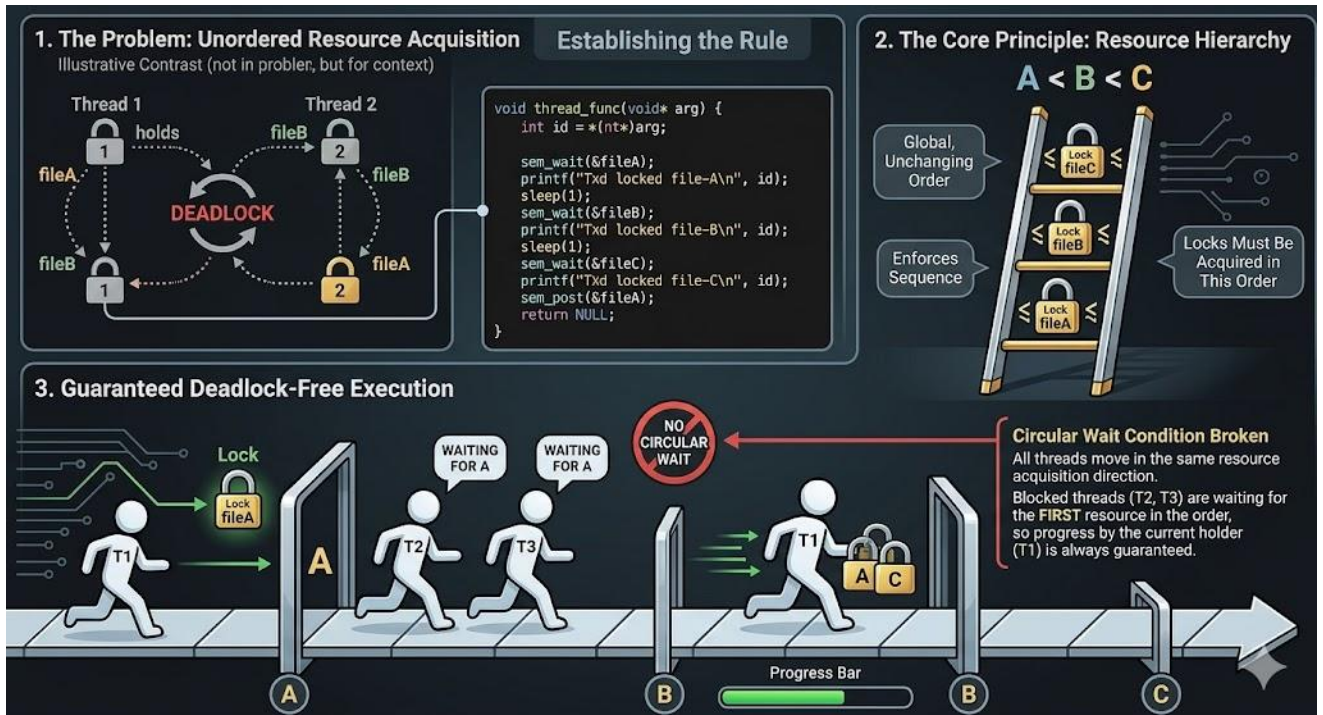
    pthread_create(&t1, NULL, thread_func, &id1);
    pthread_create(&t2, NULL, thread_func, &id2);
    pthread_create(&t3, NULL, thread_func, &id3);

    pthread_join(t1, NULL);
    pthread_join(t3, NULL);

    sem_destroy(&fileA);
    sem_destroy(&fileB);
    sem_destroy(&fileC);
    return 0;
}
    
```

National University of Computer and Emerging Sciences

Karachi Campus



b) A system has 3 resource types and 5 processes.

Process	Allocation			Max Need		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

The current allocation state is: **Available = (3,3,2)**

- Compute the availability of resources and remaining need, using Banker's Algorithm. [8]
- Determine if the process's remaining needs are in safe sequence or not? And either system is in a safe state. [2]

National University of Computer and Emerging Sciences

Karachi Campus

BANKER'S ALGORITHM – MULTIPLE SOLUTIONS

System has 3 resource types (A, B, C) and 5 processes (P0 – P4)
Initial Available = (3, 3, 2)

1. GIVEN DATA

Process	Allocation			Max Need		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

2. NEED MATRIX (Need = Max Need – Allocation)

Process	Need (A B C)		
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

REMAINING NEED MATRIX (Same as above)

P0 : (7, 4, 3) P1 : (1, 2, 2)
P2 : (6, 0, 0) P3 : (0, 1, 1)
P4 : (4, 3, 1)

KEY:

- Need ≤ Available? → If Yes, process can execute.
- After execution, release its Allocation and update Available.

3. SAFE SEQUENCE – SOLUTION 1

P1 → P3 → P0 → P2 → P4

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P1	(1, 2, 2) ≤ (3, 3, 2)	(3, 3, 2)	(2, 0, 0)	(5, 3, 2)
2	P3	(0, 1, 1) ≤ (5, 3, 2)	(5, 3, 2)	(2, 1, 1)	(7, 4, 3)
3	P0	(7, 4, 3) ≤ (7, 4, 3)	(7, 4, 3)	(0, 1, 0)	(7, 5, 3)
4	P2	(6, 0, 0) ≤ (7, 5, 3)	(7, 5, 3)	(3, 0, 2)	(10, 5, 5)
5	P4	(4, 3, 1) ≤ (10, 5, 5)	(10, 5, 5)	(0, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

4. SAFE SEQUENCE – SOLUTION 2

P1 → P3 → P2 → P0 → P4

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P1	(1, 2, 2) ≤ (3, 3, 2)	(3, 3, 2)	(2, 0, 0)	(5, 3, 2)
2	P3	(0, 1, 1) ≤ (5, 3, 2)	(5, 3, 2)	(2, 1, 1)	(7, 4, 3)
3	P2	(6, 0, 0) ≤ (7, 4, 3)	(7, 4, 3)	(3, 0, 2)	(10, 4, 5)
4	P0	(7, 4, 3) ≤ (10, 4, 5)	(10, 4, 5)	(0, 1, 0)	(10, 5, 5)
5	P4	(4, 3, 1) ≤ (10, 5, 5)	(10, 5, 5)	(0, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

5. SAFE SEQUENCE – SOLUTION 3

P3 → P1 → P2 → P0 → P4

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P3	(0, 1, 1) ≤ (3, 3, 2)	(3, 3, 2)	(2, 1, 1)	(5, 4, 3)
2	P1	(1, 2, 2) ≤ (5, 4, 3)	(5, 4, 3)	(2, 0, 0)	(7, 4, 3)
3	P2	(6, 0, 0) ≤ (7, 4, 3)	(7, 4, 3)	(3, 0, 2)	(10, 4, 5)
4	P0	(7, 4, 3) ≤ (10, 4, 5)	(10, 4, 5)	(0, 1, 0)	(10, 5, 5)
5	P4	(4, 3, 1) ≤ (10, 5, 5)	(10, 5, 5)	(0, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

6. SAFE SEQUENCE – SOLUTION 4

P3 → P1 → P4 → P0 → P2

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P3	(0, 1, 1) ≤ (3, 3, 2)	(3, 3, 2)	(2, 1, 1)	(5, 4, 3)
2	P1	(1, 2, 2) ≤ (5, 4, 3)	(5, 4, 3)	(2, 0, 0)	(7, 4, 3)
3	P4	(4, 3, 1) ≤ (7, 4, 3)	(7, 4, 3)	(0, 0, 2)	(7, 4, 5)
4	P0	(7, 4, 3) ≤ (7, 4, 5)	(7, 4, 5)	(0, 1, 0)	(7, 5, 5)
5	P2	(6, 0, 0) ≤ (7, 5, 5)	(7, 5, 5)	(3, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

7. SAFE SEQUENCE – SOLUTION 5

P3 → P1 → P0 → P2 → P4

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P3	(0, 1, 1) ≤ (3, 3, 2)	(3, 3, 2)	(2, 1, 1)	(5, 4, 3)
2	P1	(1, 2, 2) ≤ (5, 4, 3)	(5, 4, 3)	(2, 0, 0)	(7, 4, 3)
3	P0	(7, 4, 3) ≤ (7, 4, 3)	(7, 4, 3)	(0, 1, 0)	(7, 5, 3)
4	P2	(6, 0, 0) ≤ (7, 5, 3)	(7, 5, 3)	(3, 0, 2)	(10, 5, 5)
5	P4	(4, 3, 1) ≤ (10, 5, 5)	(10, 5, 5)	(0, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

8. SAFE SEQUENCE – SOLUTION 6 (Included as requested)

P1 → P3 → P4 → P0 → P2

Step	Process Executed	Need (A B C)	Available Before	Release Allocation (A B C)	Available After
1	P1	(1, 2, 2) ≤ (3, 3, 2)	(3, 3, 2)	(2, 0, 0)	(5, 3, 2)
2	P3	(0, 1, 1) ≤ (5, 3, 2)	(5, 3, 2)	(2, 1, 1)	(7, 4, 3)
3	P4	(4, 3, 1) ≤ (7, 4, 3)	(7, 4, 3)	(0, 0, 2)	(7, 4, 5)
4	P0	(7, 4, 3) ≤ (7, 4, 5)	(7, 4, 5)	(0, 1, 0)	(7, 5, 5)
5	P2	(6, 0, 0) ≤ (7, 5, 5)	(7, 5, 5)	(3, 0, 2)	(10, 5, 7)

All processes can finish → System is in **SAFE STATE**

CONCLUSION

All the above sequences allow every process to complete successfully.
Therefore,
the system is in a **SAFE STATE**.

SUMMARY OF SAFE SEQUENCES

- 1) P1 → P3 → P0 → P2 → P4
- 2) P1 → P3 → P2 → P0 → P4
- 3) P3 → P1 → P2 → P0 → P4
- 4) P3 → P1 → P4 → P0 → P2
- 5) P3 → P1 → P0 → P2 → P4
- 6) P1 → P3 → P4 → P0 → P2

National University of Computer and Emerging Sciences

Karachi Campus

CLO # 5 - Understand the deadlock and memory management.

Q3.[a(6)+b(6)+c(6)+d(6) = 24 Points | 40 Mins]

An AI research firm has developed the "Titan-II Lite." The architecture utilizes a 32-bit Logical Address Space and a 2-level Hierarchical Paging scheme.

System Specifications:

- Logical Address: 32 bits.
- Page Size: 4 KB (2^{12} bytes).
- PTE (Page Table Entry) Size: 4 bytes (2^2 bytes).
- Logical Address Split: The Page Number bits are split between the Outer Page Table (L1) and the Inner Page Table (L2) such that the Outer Page Table fits exactly within a single page frame.
- Memory Hierarchy:
 - TLB: 95% hit rate, 1 ns access time.
 - Main Memory: 50 ns access time.
- Assume: A 2-level page table walk requires 2 main memory accesses (one for L1, one for L2).

a) For Address Structure compute the following

- i. Determine the number of bits used for the Page Offset. **[2]**
- ii. Calculate the number of bits for the Outer Page Table (L1) and the Inner Page Table (L2) based on the requirement that L1 fits in one page. **[2]**
- iii. How many entries are in the Inner Page Table (L2)? **[2]**

b) For Memory Overhead

- i. Calculate the total size (in bytes) of the Outer Page Table (L1). **[3]**
- ii. If a process requires 16 MB of contiguous memory, how many Inner Page Tables (L2) are required to map this space? **[3]**

c) Calculate the Effective Memory Access Time (EMAT) for this system. **[6]**

Hint: Include the calculation for a TLB hit versus a TLB miss (where the hardware must perform the 2-level walk).

d) The CPU generates Logical Address 0x00403ABC.

- i. Extract the L1 Index **[2]**
- ii. Extract the L2 Index **[2]**
- iii. Extract the Page Offset **[2]**

National University of Computer and Emerging Sciences Karachi Campus

Q3. (3×4 = 12 Marks | 40 Mins)

An AI research firm has developed the "Titan-II Lite." The architecture utilizes a 32-bit Logical Address Space and a 2-level Hierarchical Paging scheme.

System Specifications

- Logical Address: 32 bits
- Page Size: 4 KB (2^{12} bytes)
- PTE (Page Table Entry) Size: 4 bytes (2^2 bytes)
- Logical Address Split: The Page Number bits are split between the Outer Page Table (L1) and the Inner Page Table (L2) such that the Outer Page Table fits exactly within a single page frame.

Memory Hierarchy

Assume: A 2-level page table walk requires 2 main memory accesses (one for L1, one for L2).

(a) Address Structure

- Page Offset bits**
Page size = 4 KB = 2^{12} bytes $\Rightarrow$ **Page Offset = 12 bits**
- Total Page Number bits**
32 (total) - 12 (offset) = 20 bits
- L1 and L2 split (L1 fits in one page)**
Each page table fits in one page (4 KB)
Entries per page table = $4096 / 4 = 1024 = 2^{10}$ entries
 $\Rightarrow$ L2 index bits = 10 bits
Remaining for L1 = $20 - 10 = 10$ bits
- Number of entries in L2**
Entries in L2 = $2^{10} = 1024$ entries

32-bit Logical Address

L1 Index (10 bits)	L2 Index (10 bits)	Page Offset (12 bits)
31 22	12	0

Summary (a)

- Page Offset = 12 bits
- L1 Index = 10 bits
- L2 Index = 10 bits
- Entries in L2 = 1024

(b) Memory Overhead

- Size of Outer Page Table (L1)**

Item	Value
Number of entries in L1	$2^{10} = 1024$
Size of each PTE	4 bytes
Total size of L1	$1024 \times 4 = 4096$ bytes = 4 KB
- Number of Inner Page Tables (L2) needed for 16 MB**
Process requires 16 MB contiguous memory.

Item	Value
Memory required	16 MB = $16 \times 2^{20} = 2^{24}$ bytes
Page size	2^{12} bytes
Number of pages	$2^{24} / 2^{12} = 2^{12} = 4096$ pages
Pages mapped by one L2	$1024 = 2^{10}$ pages
Number of L2 tables required	$4096 / 1024 = 4$

Summary (b)

- Size of L1 = 4096 bytes = 4 KB
- L2 tables required for 16 MB = 4

(c) Effective Memory Access Time (EMAT)

Given:

- TLB hit rate = 95%
- TLB access time = 1 ns
- Main memory access time = 50 ns
- A 2-level page table walk requires 2 main memory accesses (L1 + L2)

- TLB Hit (Probability = 95%)**

TLB Lookup
(1 ns)

→

Data Access
(50 ns)

$T_{hit} = 1 + 50 = 51$ ns
- TLB Miss (Probability = 5%)**

TLB Lookup
(1 ns)

→

L1 Access
(50 ns)

→

L2 Access
(50 ns)

→

Data Access
(50 ns)

$T_{miss} = 1 + 50 + 50 + 50 = 151$ ns

EMAT Calculation

EMAT = (Hit Rate $\times$ T_{hit}) + (Miss Rate $\times$ T_{miss})

= $(0.95 \times 51) + (0.05 \times 151)$

= 48.45 + 7.55

= **56 ns**

(d) Address Breakdown for 0x00403 ABC

- 32-bit address in Binary**

Hex	0	0	4	0	3	A	B	C
Binary (4 bits each)	0000	0000	0100	0000	0011	1010	1011	1100

Bit Positions: 31 22 21 12 11 0

L1 Index
(10 bits)

L2 Index
(10 bits)

Page Offset
(12 bits)
- Extract each field**

L1 Index (bits 31-22)
Bits [31:22] = 0000 0000 01
= **0x001**
= 1 (decimal)

L2 Index (bits 21-12)
Bits [21:12] = 0000 0011 1010
= **0x03A**
= 58 (decimal)

Page Offset (bits 11-0)
Bits [11:0] = 1010 1011 1100
= **0xABC**
= 2748 (decimal)

Summary (d)

- L1 Index = 0x001
- L2 Index = 0x03A
- Page Offset = 0xABC

CLO # 2 - Design and Implement programs using processes and threads.

Q4.[15 Points | 30 Mins]

a) A university system uses two processes to handle student marks:

Thread-1 (Writer): stores marks of 5 students in shared memory,

Thread-2 (Analyzer): reads the marks and computes total and average. The Analyzer must read the data only after all marks have been written.

National University of Computer and Emerging Sciences

Karachi Campus

Write a shared memory-based pseudo-code solution using appropriate system calls that

- Defines an appropriate shared memory structure [5]
- Implements the Writer thread and the Analyzer thread. [7]
- Write a code snippet that demonstrates the use of a suitable synchronization tool in shared memory between two threads. [3]

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <fcntl.h>
6  #include <sys/mman.h>
7  #include <sys/stat.h>
8  #include <unistd.h>
9
10 #define STUDENTS 5
11 #define SHM_NAME "/student_marks_shm"
12
13 // Shared memory structure
14 typedef struct {
15     int marks[STUDENTS];
16     int is_written;
17 } SharedData;
18
19 SharedData *data;
20 sem_t sync_sem;
21
22 // ----- Writer Thread -----
23 void* writerThread(void* arg)
24 {
25     printf("Writer: Entering marks...\n");
26     for (int i = 0; i < STUDENTS; i++)
27     {
28         printf("Enter mark of student %d: ", i + 1);
29         scanf("%d", &data->marks[i]);
30     }
31     data->is_written = 1;
32     printf("Writer: Data written successfully.\n");
33     // Signal analyzer
34     sem_post(&sync_sem);
35     return NULL;
36 }
37 // ----- Analyzer Thread -----
38 void* analyzerThread(void* arg)
39 {
40     int total = 0;
41     printf("Analyzer: Waiting for data...\n");
42     // Wait until writer completes
43     sem_wait(&sync_sem);
44     printf("Analyzer: Reading data...\n");
45     for (int i = 0; i < STUDENTS; i++)
46     {
47         total += data->marks[i];
48     }
49     float avg = total / (float)STUDENTS;
50     printf("Total Marks = %d\n", total);
51     printf("Average Marks = %.2f\n", avg);
52     return NULL;
53 }
```

National University of Computer and Emerging Sciences

Karachi Campus

```
67 // ----- Main Function -----
68 int main()
69 {
70     int shm_fd;
71     pthread_t writer, analyzer;
72
73     // Create shared memory
74     shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
75     if (shm_fd == -1)
76     {
77         perror("shm_open failed");
78         exit(1);
79     }
80
81     // Set size of shared memory
82     ftruncate(shm_fd, sizeof(SharedData));
83
84     // Map shared memory
85     data = (SharedData*) mmap(0, sizeof(SharedData),
86                               PROT_READ | PROT_WRITE,
87                               MAP_SHARED, shm_fd, 0);
88
89     if (data == MAP_FAILED)
90     {
91         perror("mmap failed");
92         exit(1);
93     }
94
95     // Initialize shared memory
96     data->is_written = 0;
97     // Initialize semaphore (shared between threads)
98     sem_init(&sync_sem, 0, 0); // 0 = initial value (Analyzer waits)
99
100    // Create threads
101    pthread_create(&writer, NULL, writerThread, NULL);
102    pthread_create(&analyzer, NULL, analyzerThread, NULL);
103
104    // Wait for threads to finish
105    pthread_join(writer, NULL);
106    pthread_join(analyzer, NULL);
107
108    // Cleanup
109    sem_destroy(&sync_sem);
110    munmap(data, sizeof(SharedData));
111    shm_unlink(SHM_NAME);
112    printf("Program finished successfully.\n");
113 }
114 return 0;
```

CLO # 3 Evaluate the commonly used mechanism for scheduling tasks.

Q5. [6 Points | 8 Mins]

A system schedules the following processes: By using the MLFQ algorithm, answer the questions given below;

P	AT	BT	Priority
P1	0	6	2
P2	3	10	4
P3	1	3	1
P4	2	5	3
P5	6	7	5
P6	4	2	1

Queue1 uses Round Robin ($q = 2$ ms), Queue2 uses Preemptive Priority (lower value = higher priority), and Queue3 uses non-preemptive SJF. A process in Q2 is demoted to Q3 only if it consumes more than 4 ms of CPU time without finishing. Context switching time is 1 ms. No aging is applied.

- Draw the Gantt chart (show preemptions and demotions). **[3]**

National University of Computer and Emerging Sciences

Karachi Campus

- ii. Compute waiting time and turnaround time for each process. [2]
- iii. Which process has the best response time? Justify briefly. [1]

Queue 1. RR $\rightarrow$ Quantum = 2

$\rightarrow$

P ₁	CS	P ₃	CS	P ₄	CS	P ₂	CS	P ₆	CS	P ₅
0	2	3	5	6	8	9	11	12	14	15
									X	

Queue 2 Priority if above 4ms, demote to Q₃

~~| | | | | | | | | |
|----------------|----|----------------|----|----------------|----|----------------|----|----------------|
| P ₃ | CS | P ₁ | CS | P ₄ | CS | P ₂ | CS | P ₅ |
| 17 | 18 | 19 | 23 | 24 | 27 | 28 | 32 | 33 |
| | X | | X | | X | | X | |~~

$\rightarrow$

P ₃	CS	P ₁	CS	P ₄	CS	P ₂	CS	P ₅
17	18	19	23	24	27	28	32	33
	X		X		X			

P₂ and P₅ demoted to Queue 3 as per rule

$\rightarrow$ Queue 3

P ₅	CS	P ₂
37	38	43
	X	X

$\rightarrow$	Completion Time	TAT	WT
P ₁ $\Rightarrow$	23	P ₁ $\Rightarrow$ 23	P ₁ $\Rightarrow$ 17
P ₂ $\Rightarrow$	43	P ₂ $\Rightarrow$ 40	P ₂ $\Rightarrow$ 30
P ₃ $\Rightarrow$	18	P ₃ $\Rightarrow$ 17	P ₃ $\Rightarrow$ 14
P ₄ $\Rightarrow$	27	P ₄ $\Rightarrow$ 25	P ₄ $\Rightarrow$ 20
P ₅ $\Rightarrow$	38	P ₅ $\Rightarrow$ 32	P ₅ $\Rightarrow$ 25
P ₆ $\Rightarrow$	14	P ₆ $\Rightarrow$ 10	P ₆ $\Rightarrow$ 8

Best Response Time

(P₁) $\rightarrow$ It arrived first
cpu started executing it
immediately.

National University of Computer and Emerging Sciences

Karachi Campus

CLO # 1 - Understand mechanisms in modern operating systems

Q6. [a(3)+b(3) = 6 Points | 7 Mins]

- a) You are developing a high-performance, complex file management utility for a CPU architecture that has only 8 general-purpose registers available for parameter passing. During a specific operation, your program needs to request an advanced system call from the operating system kernel that requires exactly 12 distinct arguments—including memory pointers, hardware flags, sector offsets, and security permissions. Identify the most suitable method to pass these 12 parameters to the operating system, and explain why you chose this method over the other two alternatives. **[3]**

The most suitable method is to pass all 12 arguments using a memory block (structure) and pass its pointer in a register to the kernel. This is best because the CPU has only 8 registers, so register-based passing is insufficient. Stack passing is possible but adds overhead and complexity. A memory block approach is scalable, efficient, and commonly used for complex system calls.

- b) A real-time data analytics system is deployed in a smart dairy farm (monitoring cattle health). The system continuously collects sensor data (temperature, movement, heartbeat) and stores it in main memory for processing.

The system has the following characteristics:

- Sensor module generates large chunks of data (16 MB every few seconds)
- CPU is already heavily utilized running AI-based prediction algorithms
- The system must ensure minimal delay in processing incoming data
- Two data transfer options are available:

- CPU-driven I/O (Programmed I/O)
- Direct Memory Access (DMA)

- i. Which data transfer method (DMA or CPU-driven I/O) would you choose for this system? Justify your answer based on the scenario. **[1.5]**

DMA is the best choice because it transfers large sensor data (16 MB chunks) directly to memory without CPU involvement, which reduces CPU overhead. Since the CPU is already heavily loaded with AI processing and low latency is required, DMA allows data transfer and computation to overlap efficiently

- ii. Explain what would happen to system performance if the other method (the one you did not choose) is used instead. **[1.5]**

DMA is the best choice because it transfers large sensor data (16 MB chunks) directly to memory without CPU involvement, which reduces CPU overhead. Since the CPU is already heavily loaded with AI processing and low latency is required, DMA allows data transfer and computation to overlap efficiently

*** Good Luck ***

National University of Computer and Emerging Sciences

Karachi Campus

Q5 part a , part iii Another Alternative Solution

This can also be correct

C PROGRAM (Deadlock-Free Version with Global Lock Order)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  // Define semaphores representing the files
8  sem_t file_A;
9  sem_t file_B;
10 sem_t file_C;
11
12 // Thread 1: Locks A, then B
13 void* thread1_func(void* arg) {
14     printf("[Thread 1] Locking File A...\n");
15     sem_wait(&file_A);
16     printf("[Thread 1] Locked File A. Working...\n");
17     sleep(1);
18
19     printf("[Thread 1] Locking File B...\n");
20     sem_wait(&file_B);
21     printf("[Thread 1] Locked File B. Processing...\n");
22     sleep(1);
23
24     sem_post(&file_B);
25     sem_post(&file_A);
26     printf("[Thread 1] Released File C and File A.\n");
27     return NULL;
28 }
29
30 // Thread 2: Locks B, then C
31 void* thread2_func(void* arg) {
32     printf("[Thread 2] Locking File B...\n");
33     sem_wait(&file_B);
34     printf("[Thread 2] Locked File B. Working...\n");
35     sleep(1);
36
37     printf("[Thread 2] Locking File C...\n");
38     sem_wait(&file_C);
39     printf("[Thread 2] Locked File C. Processing...\n");
40     sleep(1);
41
42     sem_post(&file_C);
43     sem_post(&file_B);
44     printf("[Thread 2] Released File C and File A.\n");
45     return NULL;
46 }
47
48 // Thread 3: Locks A, then C (Modified: A before C)
49 void* thread3_func(void* arg) {
50     printf("[Thread 3] Locking File A...\n");
51     sem_wait(&file_A);
52     printf("[Thread 3] Locked File A. Working...\n");
53     sleep(1);
54
55     printf("[Thread 3] Locking File C...\n");
56     sem_wait(&file_C);
57     printf("[Thread 3] Locked File C. Processing...\n");
58     sleep(1);
59
60     sem_post(&file_C);
61     sem_post(&file_A);
62     printf("[Thread 3] Released File C and File A.\n");
63     return NULL;
64 }
65
66 int main() {
67     pthread_t t1, t2, t3;
68
69     sem_init(&file_A, 0, 1);
70     sem_init(&file_B, 0, 1);
71     sem_init(&file_C, 0, 1);
72
73     printf("=== Starting Deadlock-Free Execution ===\n");
74
75     pthread_create(&t1, NULL, thread1_func, NULL);
76     pthread_create(&t2, NULL, thread2_func, NULL);
77     pthread_create(&t3, NULL, thread3_func, NULL);
78
79     pthread_join(t1, NULL);
80     pthread_join(t2, NULL);
81     pthread_join(t3, NULL);
82
83     sem_destroy(&file_A);
84     sem_destroy(&file_B);
85     sem_destroy(&file_C);
86
87     printf("=== All threads completed successfully ===\n");
88     return 0;
89 }

```

Deadlock Prevention by Enforcing a Global Lock Order

Instead of letting threads lock files in a circular manner, we establish a **global order** for acquiring locks. In this case, we dictate that files must always be locked in alphabetical order (**File A** → **File B** → **File C**).



- Thread 1:** Locks A, then B (Already follows alphabetical order)
- Thread 2:** Locks B, then C (Already follows alphabetical order)
- Thread 3:** Originally wanted C then A. We modify it to lock A first, then C.



By making Thread 3 lock File A before File C, we completely eliminate the cycle. If Thread 1 grabs File A, Thread 3 will be forced to wait at the very beginning rather than holding File C and blocking Thread 2.

HOW THIS RESOLVES THE DEADLOCK

1. Whichever thread targets File A first (either Thread 1 or Thread 3) will claim it.

Thread 1

OR

Thread 3
2. Let's say Thread 1 wins and locks File A.

Thread 1

A → B → C
3. When Thread 3 starts, it tries to lock File A, finds it occupied, and safely blocks without holding any other resources.

Thread 3

→ A (waiting)
4. Because Thread 3 is blocked at the gate and hasn't locked File C yet, Thread 2 is free to lock File B, then successfully lock File C, finish its job, and release them both.

Thread 2

→ B → C
5. Once the resources start cascading open, Thread 1 finishes, which finally frees up File A for Thread 3 to do its work.

Thread 1 done

A → Thread 3 proceeds

A → C

done

The execution loop completes cleanly, and the program terminates on its own.